

Лабораторная работа по ML №2 Жуховицки

Датасет:

- [Telco Customer Churn](#)

Загрузить и изучить датасет:

- Провести EDA: типы признаков, распределения, корреляции.
- Очистить данные (удалить/заменить пропуски, привести типы, закодировать категориальные признаки где это требуется по особенностям модели).
- Разделить данные на обучающую и тестовую выборки.
- Обучить модели:
 - RandomForestClassifier
 - Градиентный бустинг (любой на выбор, не из sklearn)
 - SVC (classifier SVM)
 - KNeighborsClassifier
- Оценить качество моделей с помощью:
 - Accuracy,
 - Precision,
 - Recall,
 - F1-score,
 - ROC-AUC.
- Построить матрицу ошибок и ROC-кривую.
- Сделать вывод: какая модель показывает наилучший баланс метрик

Стадия preprocessing'a

Получение и чистка данных

```
from kagglehub import KaggleDatasetAdapter, dataset_load

df = dataset_load(
    KaggleDatasetAdapter.PANDAS,
    "blastchar/telco-customer-churn",
    path="WA_Fn-UseC_-Telco-Customer-Churn.csv",
)

print(df.info())
df
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customerID            7043 non-null   object
1   gender                 7043 non-null   object
2   SeniorCitizen          7043 non-null   int64
3   Partner                7043 non-null   object
4   Dependents             7043 non-null   object
5   tenure                 7043 non-null   int64
6   PhoneService           7043 non-null   object
7   MultipleLines           7043 non-null   object
8   InternetService        7043 non-null   object
9   OnlineSecurity         7043 non-null   object
10  OnlineBackup           7043 non-null   object
11  DeviceProtection       7043 non-null   object
12  TechSupport            7043 non-null   object
13  StreamingTV            7043 non-null   object
14  StreamingMovies        7043 non-null   object
15  Contract               7043 non-null   object
16  PaperlessBilling       7043 non-null   object
17  PaymentMethod          7043 non-null   object
18  MonthlyCharges         7043 non-null   float64
19  TotalCharges           7043 non-null   object
20  Churn                  7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
None

```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLi
0	7590-VHVEG	Female	0	Yes	No	1	No	No ph ser
1	5575-GNVDE	Male	0	No	No	34	Yes	
2	3668-QPYBK	Male	0	No	No	2	Yes	
3	7795-CFOCW	Male	0	No	No	45	No	No ph ser
4	9237-HQITU	Female	0	No	No	2	Yes	
...	...	...	...	...	...	...	...	
7038	6840-RESVB	Male	0	Yes	Yes	24	Yes	
7039	2234-XADUH	Female	0	Yes	Yes	72	Yes	
7040	4801-JZAZL	Female	0	Yes	Yes	11	No	No ph ser
7041	8361-LTMKD	Male	1	Yes	No	4	Yes	
7042	3186-AJIEK	Male	0	No	No	66	Yes	

7043 rows × 21 columns

Разберём столбцы:

- CustomerID - ID Заказчика
- Gender - Пол (бинарный строковый)
- SeniorCitizen - Пожилой ли (бинарный числовой)
- Partner - Есть ли у заказчика партнёр (бинарный строковый)
- Dependents - Есть ли зависимые от заказчика лица (бинарный строковый)
- Tenure - Знаком с компанией месяцев
- PhoneService - Есть ли мобильная связь (бинарный строковый)
- MultipleLines - Есть ли несколько линий связи (категориальный есть/нет/нет связи)
- InternetService - Провайдер (категориальный DSL/Fiber optic/No)
- OnlineSecurity - Подключена ли услуга онлайн безопасности (категориальный Yes/No/No internet service)
- OnlineBackup - Подключена ли услуга бэкапов (категориальный Yes/No/No internet service)
- DeviceProtection - Подключена ли услуга защиты устройства (категориальный Yes/No/No internet service)

- TechSupport - Использована ли услуга тех. поддержки (категориальный Yes/No/No internet service)
- StreamingTV - Подключена ли телевидение (категориальный Yes/No/No internet service)
- StreamingMovies - Подключена ли услуга онлайн-кинотеатра (категориальный Yes/No/No internet service)
- Contract - Вид договора (категориальный ежемесячный/год/2 года)
- PaperlessBilling - Безбумажное выставление счетов (бинарный строковый)
- Payment method - Вид оплаты (сложный категориальный)
- MonthlyCharges - стоимость услуг в месяц
- TotalCharges - сколько средств уже списано за срок с начала договора
- Churn - отказался ли клиент от услуг

Таргетом является столбец churn

Так как столбец с прайсом за месяц почему-то хранится в строке, обработаем столбец отдельно.

- Путём проб и ошибок установил, что в столбце есть пропущенные значения... Заменим

```
import pandas as pd

df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')
df['TotalCharges'].fillna(df['TotalCharges'].mean(), inplace=True)

df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customerID            7043 non-null   object
1   gender                 7043 non-null   object
2   SeniorCitizen          7043 non-null   int64
3   Partner                7043 non-null   object
4   Dependents             7043 non-null   object
5   tenure                 7043 non-null   int64
6   PhoneService           7043 non-null   object
7   MultipleLines           7043 non-null   object
8   InternetService         7043 non-null   object
9   OnlineSecurity          7043 non-null   object
10  OnlineBackup            7043 non-null   object
11  DeviceProtection        7043 non-null   object
12  TechSupport             7043 non-null   object
13  StreamingTV             7043 non-null   object
14  StreamingMovies         7043 non-null   object
15  Contract                7043 non-null   object
16  PaperlessBilling        7043 non-null   object
17  PaymentMethod           7043 non-null   object
18  MonthlyCharges          7043 non-null   float64
19  TotalCharges            7043 non-null   float64
20  Churn                   7043 non-null   object
dtypes: float64(2), int64(2), object(17)
memory usage: 1.1+ MB

```

Проверим пропуски

```
df.isna().sum().rename("Количество пропусков")
```

```

customerID      0
gender           0
SeniorCitizen    0
Partner          0
Dependents       0
tenure           0
PhoneService     0
MultipleLines    0
InternetService  0
OnlineSecurity   0
OnlineBackup     0
DeviceProtection 0
TechSupport      0
StreamingTV      0
StreamingMovies  0
Contract         0
PaperlessBilling 0
PaymentMethod    0
MonthlyCharges   0
TotalCharges     0
Churn            0
Name: Количество пропусков, dtype: int64

```

Рассмотрим подробнее данные и их распределение.

df								
	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLi
0	7590-VHVEG	Female	0	Yes	No	1	No	No ph ser
1	5575-GNVDE	Male	0	No	No	34	Yes	
2	3668-QPYBK	Male	0	No	No	2	Yes	
3	7795-CFOCW	Male	0	No	No	45	No	No ph ser
4	9237-HQITU	Female	0	No	No	2	Yes	
...	...	...	...	...	...	...	...	
7038	6840-RESVB	Male	0	Yes	Yes	24	Yes	
7039	2234-XADUH	Female	0	Yes	Yes	72	Yes	
7040	4801-JZAZL	Female	0	Yes	Yes	11	No	No ph ser
7041	8361-LTMKD	Male	1	Yes	No	4	Yes	
7042	3186-AJIEK	Male	0	No	No	66	Yes	

7043 rows × 21 columns

Проведём энкодинг, я буду использовать LabelEncoding для колонок, которые можно свести к бинарным, и OneHotEncoding для категориальных признаков, которые нельзя привести к бинарным.

Также дропнем колонку с ID клиента, так он не вносит никакого вклада в predict. Заметим, что признаки с вариантами No/No service можно привести к бинарным.

```
# Возьмём готовые энкодеры из scikit-learn
from sklearn.preprocessing import LabelEncoder, OneHotEncoder

if "customerID" in df.columns:
    df.drop(columns=["customerID"], inplace=True)

le = LabelEncoder()
ohe = OneHotEncoder(sparse_output=False, drop='first')
```

```

for col in df.columns:
    if df[col].nunique() == 2:
        df[col] = le.fit_transform(df[col])
    elif "No phone service" in df[col].unique() or "No internet service" in df[col].unique():
        df[col] = df[col].map({"Yes": 1, "No": 0, "No internet service": 0, "No phone service": 0})
    elif df[col].dtype == "object":
        encoded = ohe.fit_transform(df[[col]])
        feature_names = ohe.get_feature_names_out([col])
        encoded_df = pd.DataFrame(encoded, index=df.index, columns=feature_names)
        df = df.drop(columns=[col]).join(encoded_df)

df.info()

```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 7043 entries, 0 to 7042
```

```
Data columns (total 24 columns):
```

#	Column	Non-Null Count	Dtype
0	gender	7043 non-null	int64
1	SeniorCitizen	7043 non-null	int64
2	Partner	7043 non-null	int64
3	Dependents	7043 non-null	int64
4	tenure	7043 non-null	int64
5	PhoneService	7043 non-null	int64
6	MultipleLines	7043 non-null	int64
7	OnlineSecurity	7043 non-null	int64
8	OnlineBackup	7043 non-null	int64
9	DeviceProtection	7043 non-null	int64
10	TechSupport	7043 non-null	int64
11	StreamingTV	7043 non-null	int64
12	StreamingMovies	7043 non-null	int64
13	PaperlessBilling	7043 non-null	int64
14	MonthlyCharges	7043 non-null	float64
15	TotalCharges	7043 non-null	float64
16	Churn	7043 non-null	int64
17	InternetService_Fiber optic	7043 non-null	float64
18	InternetService_No	7043 non-null	float64
19	Contract_One year	7043 non-null	float64
20	Contract_Two year	7043 non-null	float64
21	PaymentMethod_Credit card (automatic)	7043 non-null	float64
22	PaymentMethod_Electronic check	7043 non-null	float64
23	PaymentMethod_Mailed check	7043 non-null	float64

```
dtypes: float64(9), int64(15)
```

```
memory usage: 1.3 MB
```

Энкодинг завершён! Посмотрим как распределены данные!

```
df.describe()
```

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	Multi
count	7043.000000	7043.000000	7043.000000	7043.000000	7043.000000	7043.000000	7043.000000
mean	0.504756	0.162147	0.483033	0.299588	32.371149	0.903166	0.903166
std	0.500013	0.368612	0.499748	0.458110	24.559481	0.295752	0.295752
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	0.000000	0.000000	0.000000	0.000000	9.000000	1.000000	1.000000
50%	1.000000	0.000000	0.000000	0.000000	29.000000	1.000000	1.000000
75%	1.000000	0.000000	1.000000	1.000000	55.000000	1.000000	1.000000
max	1.000000	1.000000	1.000000	1.000000	72.000000	1.000000	1.000000

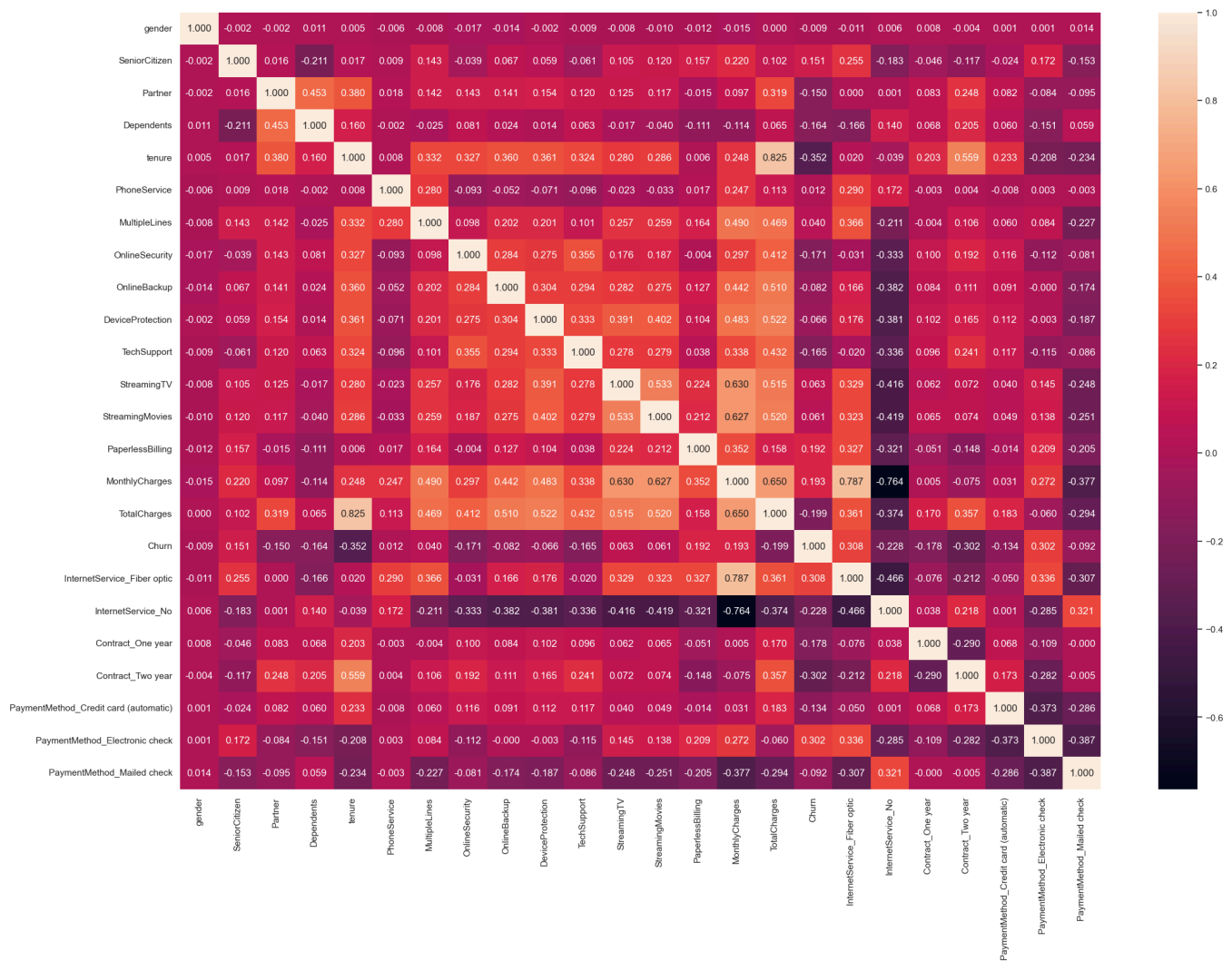
8 rows x 24 columns

Рассмотрим корреляцию признаков

```
import seaborn as sns
import matplotlib.pyplot as plt

matrix = df.corr()

plt.figure(figsize=(25,17))
sns.heatmap(matrix, annot=True, fmt=".3f")
plt.show()
```

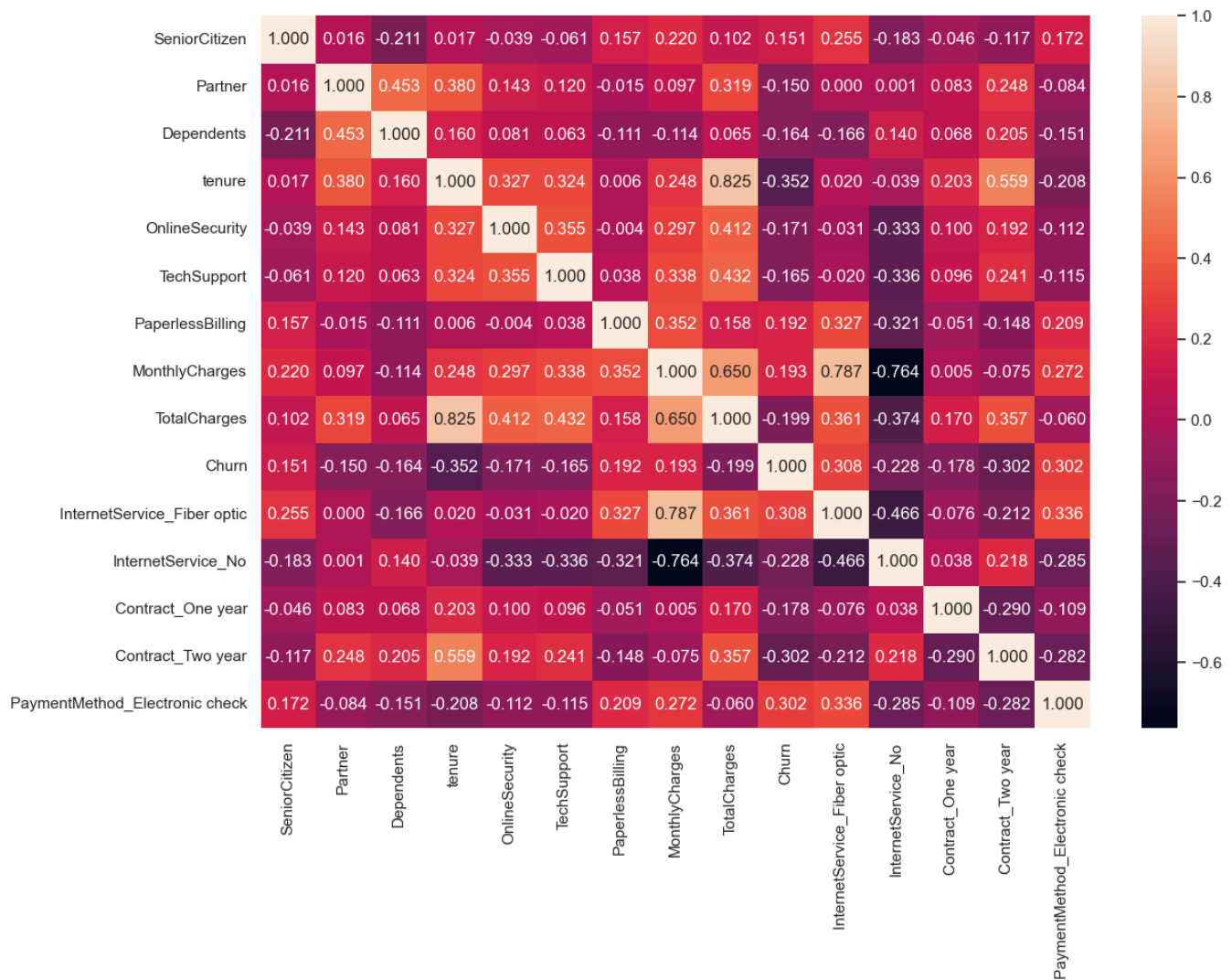



Получилось очень много колонок с низкой корреляцией с таргетом, уберём лишнее

```
churn_corr = df.corr().abs()["Churn"]
feature_names = df.columns.drop("Churn")
cols_to_drop = churn_corr[feature_names][churn_corr[feature_names] < 0.15]

df.drop(cols_to_drop.index, inplace=True, axis=1)

plt.figure(figsize=(13, 9))
sns.heatmap(df.corr(), annot=True, fmt=".3f")
plt.show()
```



Также дропнем TotalCharges, чтобы сократить количество сильно взаимоскоррелирующих признаков.

```
df.drop("TotalCharges", inplace=True, axis=1)
```

Выбранный подход энкодинга помимо своей основной задачи позволил выбросить много лишних колонок после фильтра по порогу корреляции.

Перейдём к обучению моделей:

- RandomForestClassifier
- Градиентный бустинг (любой на выбор, не из sklearn)
- SVC (classifier SVM)
- KNeighborsClassifier

```
# Импортируем все необходимые модели
```

```
# Random forest
```

```

from sklearn.ensemble import RandomForestClassifier

# GB
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier
"""Тут мог быть catboost, но он не захотел жить вместе с Python 3.14"""

# SVC
from sklearn.svm import SVC

# KNN
from sklearn.neighbors import KNeighborsClassifier

```

```

# А также импортируем всё для метрик
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, RocCurveDisplay, ConfusionMatrixDisplay
)

```

Поделим данные на train и test и создадим экземпляры наших моделей

```

from sklearn.model_selection import train_test_split

params = df.drop(columns=["Churn"])
target = df["Churn"]
params_train, params_test, target_train, target_test = train_test_split(
    params, target,
    test_size=0.2,
    stratify=target,
)

# много интересных параметров...
models = {
    "RandomForestClassifier": RandomForestClassifier(
        n_estimators=300,
        n_jobs=-1,
    ),
    "XGBClassifier": XGBClassifier(
        n_estimators=300,
        learning_rate=0.05,
        max_depth=6,
        eval_metric="auc",
        n_jobs=-1,
    ),
    "LGBMClassifier": LGBMClassifier(
        n_estimators=300,
        learning_rate=0.05,
        max_depth=6,
        class_weight="balanced",
        eval_metric="auc",
        n_jobs=-1,
        verbose=-1,
    ),
    "SVC": SVC(
        probability=True,

```

```

        kernel="rbf"
    ),
    "KNeighborsClassifier": KNeighborsClassifier(
        n_neighbors=9,
        n_jobs=-1,
    )
}

```

Обучим модели и соберём метрики, сравним их!

```

from math import ceil

results = []
target_predicts, target_probas = {}, {}

for name, model in models.items():
    model.fit(params_train, target_train)

    target_predict = model.predict(params_test)
    target_proba = model.predict_proba(params_test)[: , 1]

    target_predicts[name] = target_predict
    target_probas[name] = target_proba

    results.append(
        dict(
            Model = name,
            accuracy = accuracy_score(target_test, target_predict),
            precision = precision_score(target_test, target_predict, zero_division=0),
            recall = recall_score(target_test, target_predict, zero_division=0),
            f1 = f1_score(target_test, target_predict, zero_division=0),
            roc_auc = roc_auc_score(target_test, target_proba),
        )
    )

metrics_df = pd.DataFrame(results)

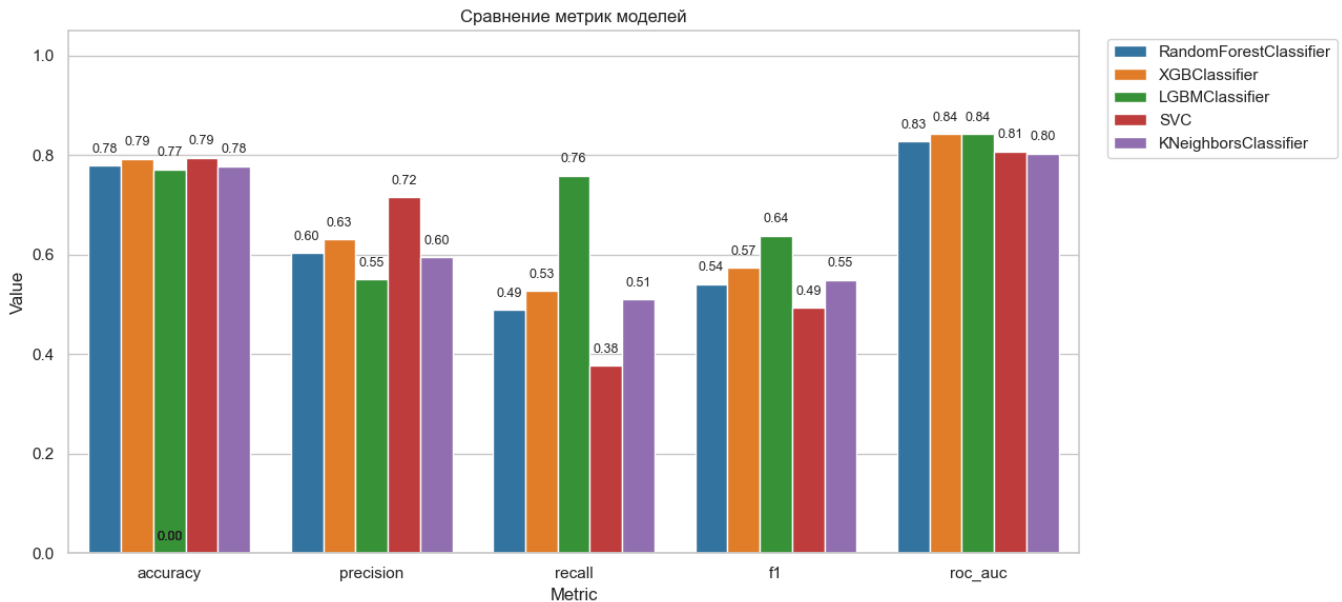
sns.set_theme(style="whitegrid", palette="tab10")
metrics_melted = metrics_df.melt(id_vars="Model", var_name="Metric", value_name="Value")

plt.figure(figsize=(13, 6))
ax = sns.barplot(
    data=metrics_melted,
    x="Metric", y="Value", hue="Model"
)

for p in ax.patches:
    height = p.get_height()
    ax.text(
        x = p.get_x() + p.get_width() / 2,
        y = height + 0.02,
        s = f"{height:.2f}",
        ha = "center", va = "bottom", fontsize=9
    )

```

```
plt.title("Сравнение метрик моделей")
plt.ylim(0, 1.05)
plt.legend(bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.show()
```



Анализ метрик показывает, что LGBM Classifier показывает превосходство при рассмотрении всех метрик, однако имеет наименьший precision, что в некоторых случаях подводит. Наиболее сбалансированно показал себя бустинг XGB, стабильно удерживающий ТОП 2.

Summary: разные градиентные бустинги удобно использовать для любых задач, так как они довольно универсальны. Для каких-то узконаправленных задач можно выбирать бустинг с отдельно настроенной конфигурацией.

В случае проблем с градиентами можно использовать и методы типа random forest или k nearest neighbors, так как они не так сильно уступают GB.

Построим матрицы ошибок и ROC-кривые, для этого используем встроенные инструменты sklearn.

Сравним Confusion matrix

```
sns.set_theme(style="white")

idx = 0
n_models = len(models)
cols = 2
rows = ceil(n_models / cols)
```

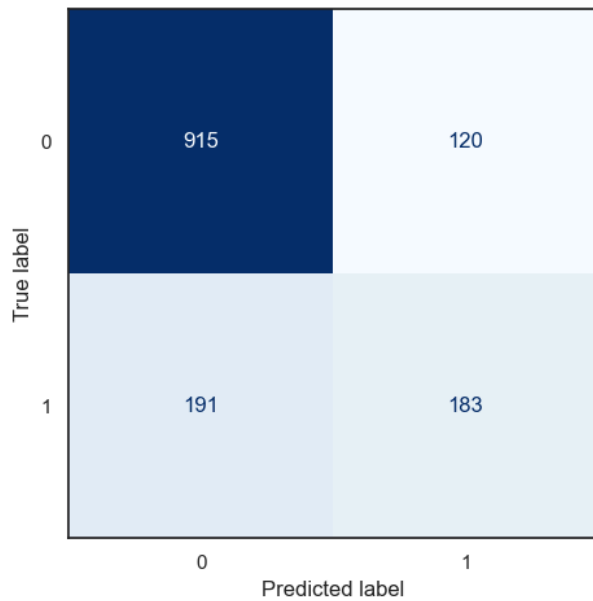
```
fig_cm, axes_cm = plt.subplots(rows, cols, figsize=(6*cols, 5*rows))
axes_cm = axes_cm.flatten()

for idx, (name, ax) in enumerate(zip(models.keys(), axes_cm)):
    ConfusionMatrixDisplay.from_predictions(
        target_test,
        target_predicts[name],
        ax=ax,
        cmap="Blues",
        colorbar=False
    )
    ax.set_title(f"{name} - Confusion Matrix")

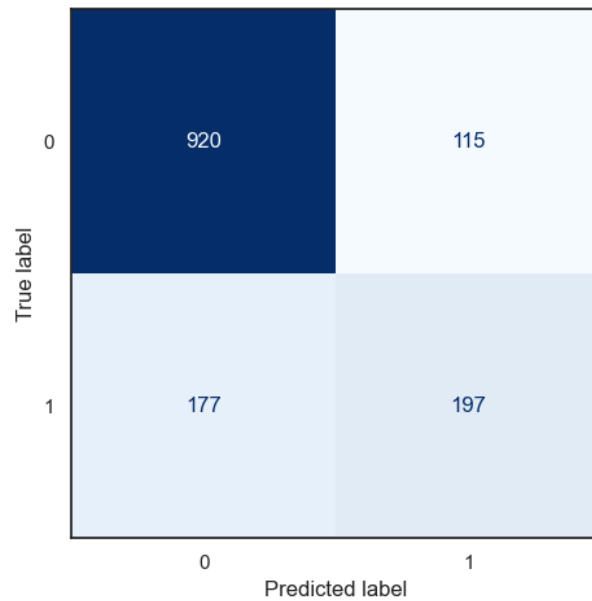
for j in range(idx + 1, len(axes_cm)):
    axes_cm[j].set_visible(False)

fig_cm.tight_layout()
```

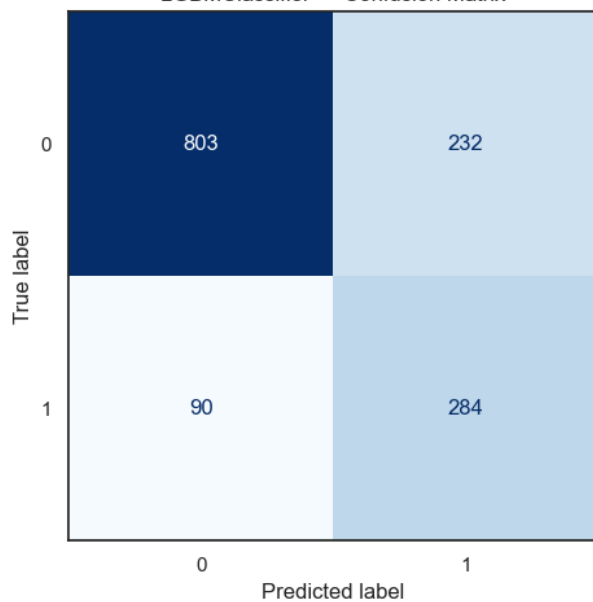
RandomForestClassifier — Confusion Matrix



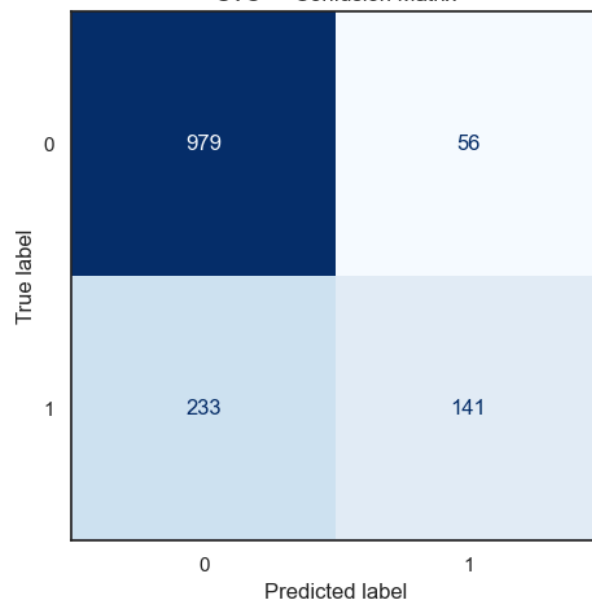
XGBClassifier — Confusion Matrix



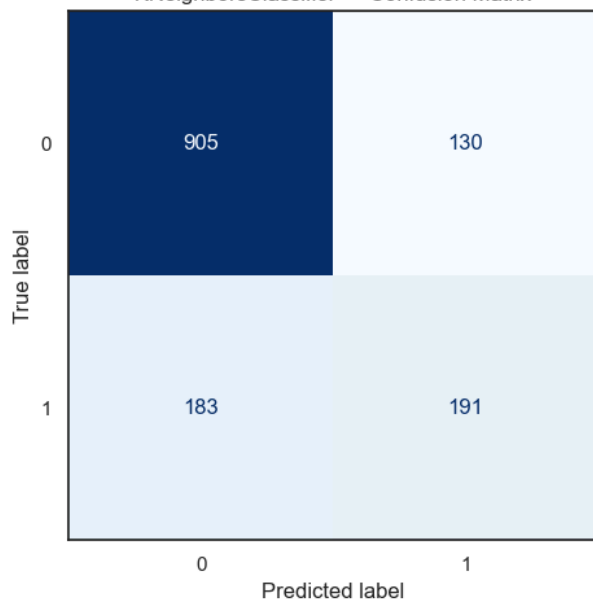
LGBMClassifier — Confusion Matrix



SVC — Confusion Matrix



KNeighborsClassifier — Confusion Matrix



Как уже можно было заметить ранее LGBM Classifier имеет высший recall, поэтому и матрица ошибок у него тоже немного отличается по распределению.

В целом можно заметить тенденцию, что все остальные модели чаще ошибаются именно на положительных значениях, что соответствует ранее рассмотренным метрикам.

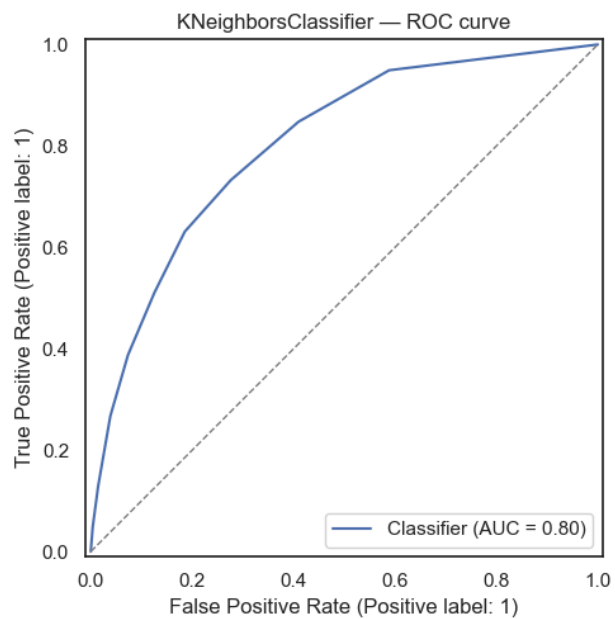
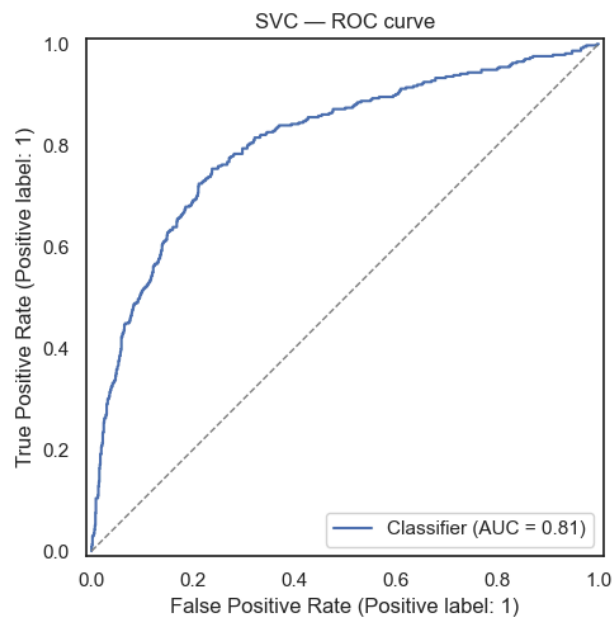
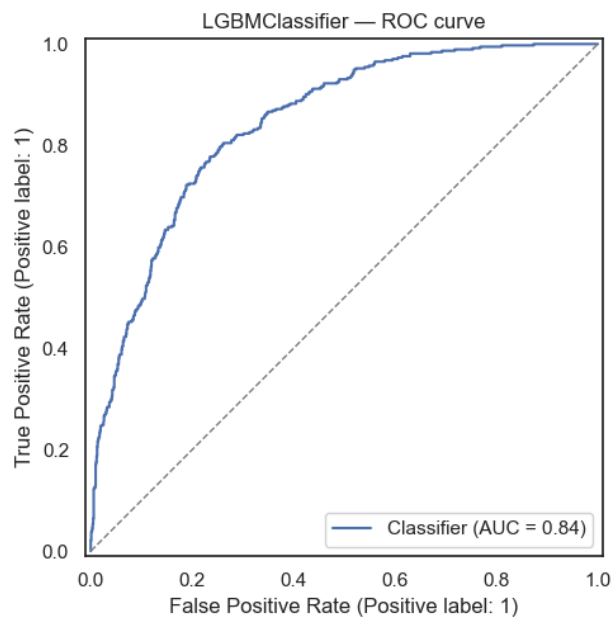
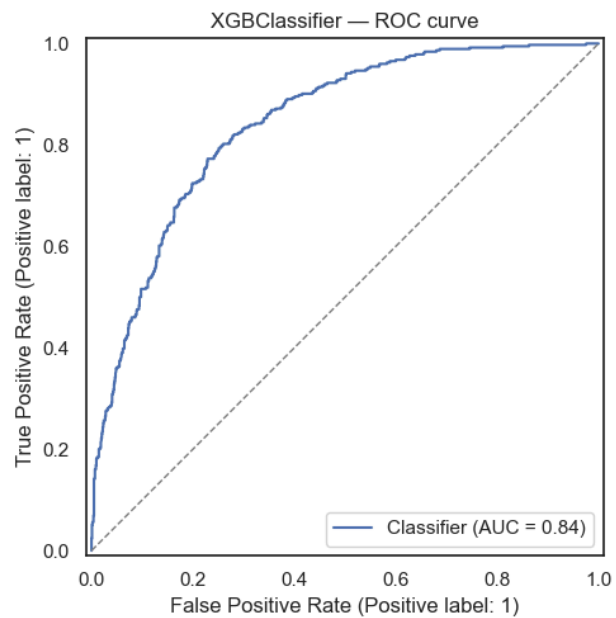
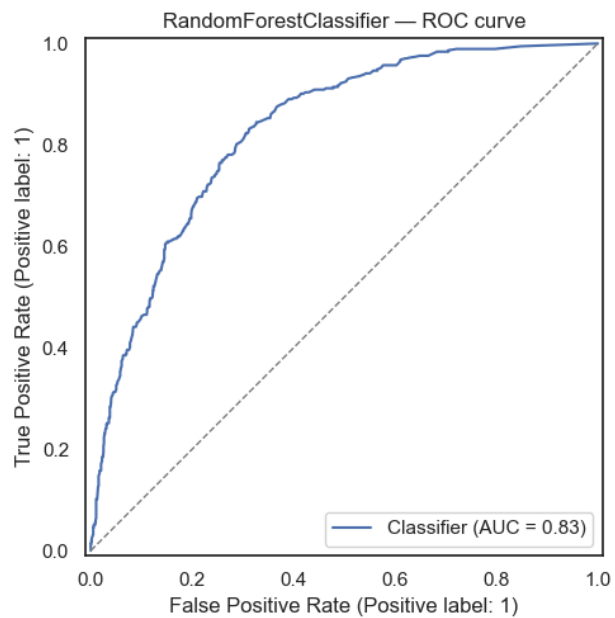
Сравним ROC curve моделей

```
fig_roc, axes_roc = plt.subplots(rows, cols, figsize=(6*cols, 5*rows))
axes_roc = axes_roc.flatten()

for idx, (name, ax) in enumerate(zip(models.keys(), axes_roc)):
    RocCurveDisplay.from_predictions(
        target_test,
        target_probas[name],
        ax=ax
    )
    ax.set_title(f"{name} - ROC curve")
    ax.plot([0, 1], [0, 1], ls="--", lw=1, c="grey")

for j in range(idx + 1, len(axes_roc)):
    axes_roc[j].set_visible(False)

fig_roc.tight_layout()
```

Как было замечено мной ранее XGB Classifier действительно показал себя оптимально и занял лидирующую позицию по AUC. Стоит также заметить, что тот же Random forest дышит в спину градиентному бустингу и тоже применим.

Выводы

Результаты показывают, что в данной задаче классификации всё-таки побеждает градиентный бустинг. XGBClassifier тренируется чуть дольше остальных, однако показывает лучшие результаты. Когда пропустить позитив дороже, чем поднять ложную тревогу лучше будет использовать LGBM: Лучший Recall (0.76), AUC 0.84; быстрее тренируется.

- *А также еще один вывод: мне надо изучать смысл метрик и метод работы моделей под*